Decorator

Decorator · Stack compression + encryption on a data store · Build with Waqar

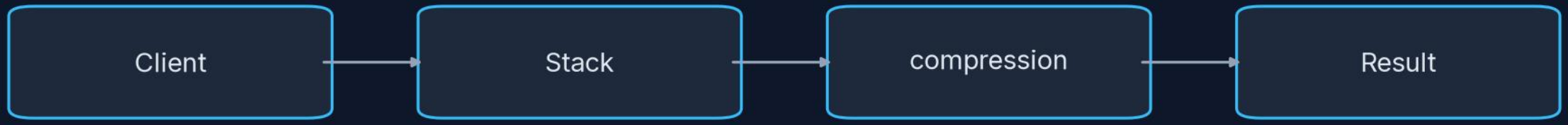
Today's Hook

- Today's force: Stack compression + encryption on a data store
- We earn Decorator by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: DataSource, InMemoryDataSource, DataSourceDecorator, EncryptionDecorator

The Problem

- Scenario: Stack compression + encryption on a data store
- Constraints: solve «Stack compression + encryption on a data store» with Decorator — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Stack compression + encryption on a data store
- Success looks like: enc(secret)

Structure



Decorator: Stack compression + encryption on a data store

Real-World Scenario

- Running example: Stack compression + encryption on a data store
- Key types: DataSource, InMemoryDataSource, DataSourceDecorator, EncryptionDecorator
- Happy-path stdout cue: enc(secret)
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.decorator.problem;

import java.util.*;

public final class NaiveStore {
  private String data="";
  public void write(String s){ data="enc("+s+")"; } // one hard-wired transform
  public String read(){ return data; }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Stack compression + encryption on a data store
- Without Decorator, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.gof.structural.decorator.problem;

import java.util.*;

interface DataSource { String read(); void write(String data); }
final class InMemoryDataSource implements DataSource {
  private String data=""; public String read(){return data;} public void write(String d){data=d;}
}
abstract class DataSourceDecorator implements DataSource {
  protected final DataSource wrappee; protected DataSourceDecorator(DataSource w){ wrappee=w; }
  public String read(){ return wrappee.read(); } public void write(String d){ wrappee.write(d); }
}
final class EncryptionDecorator extends DataSourceDecorator {
  public EncryptionDecorator(DataSource w){ super(w); }
  public void write(String d){ wrappee.write("enc("+d+")"); }
  public String read(){ String s=wrappee.read(); return s.startsWith("enc(")? s.substring(4,s.length()-1):s; }
}
final class CompressionDecorator extends DataSourceDecorator {
  public CompressionDecorator(DataSource w){ super(w); }
  public void write(String d){ wrappee.write("zip("+d+")"); }
  public String read(){ String s=wrappee.read(); return s.startsWith("zip(")? s.substring(4,s.length()-1):s; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.decorator.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Decorator: compress+encrypt store ===");
    System.out.println("--- naive ---");
    var n=new NaiveStore(); n.write("secret"); System.out.println(n.read());
    System.out.println("--- decorator stack ---");
    var mem = new InMemoryDataSource();
    DataSource ds = new EncryptionDecorator(new CompressionDecorator(mem));
    ds.write("secret");
    System.out.println("stored-view=" + mem.read());
    System.out.println("read=" + ds.read());
  }
}
```

Walkthrough Output

```
=== Decorator: compress+encrypt store ===
--- naive ---
enc(secret)
--- decorator stack ---
stored-view=zip(enc(secret))
read=secret
```

Step-by-Step Trace

- Match stdout to stages — look for: enc(secret)
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 11-decorator ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Decorator
// Prefer records + sealed where they shrink the demo.
// Problem under test: Stack compression + encryption on a data store
```

Trade-offs

- Use Decorator when «Stack compression + encryption on a data store» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Decorator in product language.

Related Patterns

- Name the pattern usually confused with Decorator and separate the forces.
- Sketch the roles for «Stack compression + encryption on a data store» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Stack compression + encryption on a data store
- Naive pain → Decorator boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.